



NEXUS

Sistema de Monitoreo Meteorológico y Gestión de Invernaderos



Nuestro Equipo

ooo

Ernesto

Database Administrator & Data
Architect

Mario

Core Engineer & Backend
Architect

Carlos

Database Administrator & Data
Architect

Diego

Business & Agronomic Rules
Analyst

Rafael

Systems Modeler & UML Architect

Iancarlo

Business & Agronomic Rules
Analyst

Gabriel

Frontend & Data Visualization
Engineer



Agenda

ooo

- 1. Fase de Descubrimiento: El Problema Agronómico**
- 2. Fase de Diseño Lógico: Arquitectura Estructural y de Procesos**
- 3. Fase de Persistencia: Ingeniería de Datos**
- 4. Fase de Construcción: Backend Services & UI Dinámica**
- 5. Fase de Despliegue: Infraestructura y Demo en Vivo**

ooo

La ingeniería **empieza**
siempre en el análisis



Preguntas clave

- ✓ ¿Por qué un pico de calor mata al cultivo?
- ✓ ¿Quiénes controlan y mitigan el riesgo?
- ✓ ¿Cómo simulamos sensores usando la NASA?

Estrés Hídrico

Cuando la temperatura del invernadero sube drásticamente o la humedad cae, las plantas sufren estrés térmico e hídrico.

Para defenderse y no deshidratarse, el cultivo activa un mecanismo de emergencia: cierra sus estomas (sus poros). Al cerrarse, la planta detiene la fotosíntesis y sufre daño celular irreversible, destruyendo la producción en cuestión de minutos.



Mitigación de Riesgos

Para garantizar la resiliencia del sistema ante cualquier contingencia climática u operativa, la arquitectura de Nexus implementa un canal doble de ingesta:

- Flujo Principal: Rutinas asíncronas en Spring Boot conectadas a la API de la NASA. Realizan descargas de telemetría cada hora sin intervención humana.
- Flujo de Contingencia: Módulo que permite inyectar datos (temperatura, humedad y radiación) directamente a los controladores.



Datos

El backend ejecuta peticiones asíncronas para extraer variables: Temperatura, Humedad y Radiación Solar



Sincronización

Ante la latencia de los datos satelitales, empatamos la estampa de tiempo local con las matrices de la NASA mediante un offset cronológico.



Procesamiento

El motor no solo persiste datos crudos; calcula diferenciales térmicos para estimar la temperatura interior del invernadero.

Reglas de Negocio

- ✓ RN-01 (Peligro Térmico) | Temperatura > 35°C
- ✓ RN-02 (Estrés Hídrico) | Humedad < 30%
- ✓ RN-03 (Inmutabilidad de Incidencias): Toda alerta nace "Activa" y requiere auditoría y resolución manual.

ooo

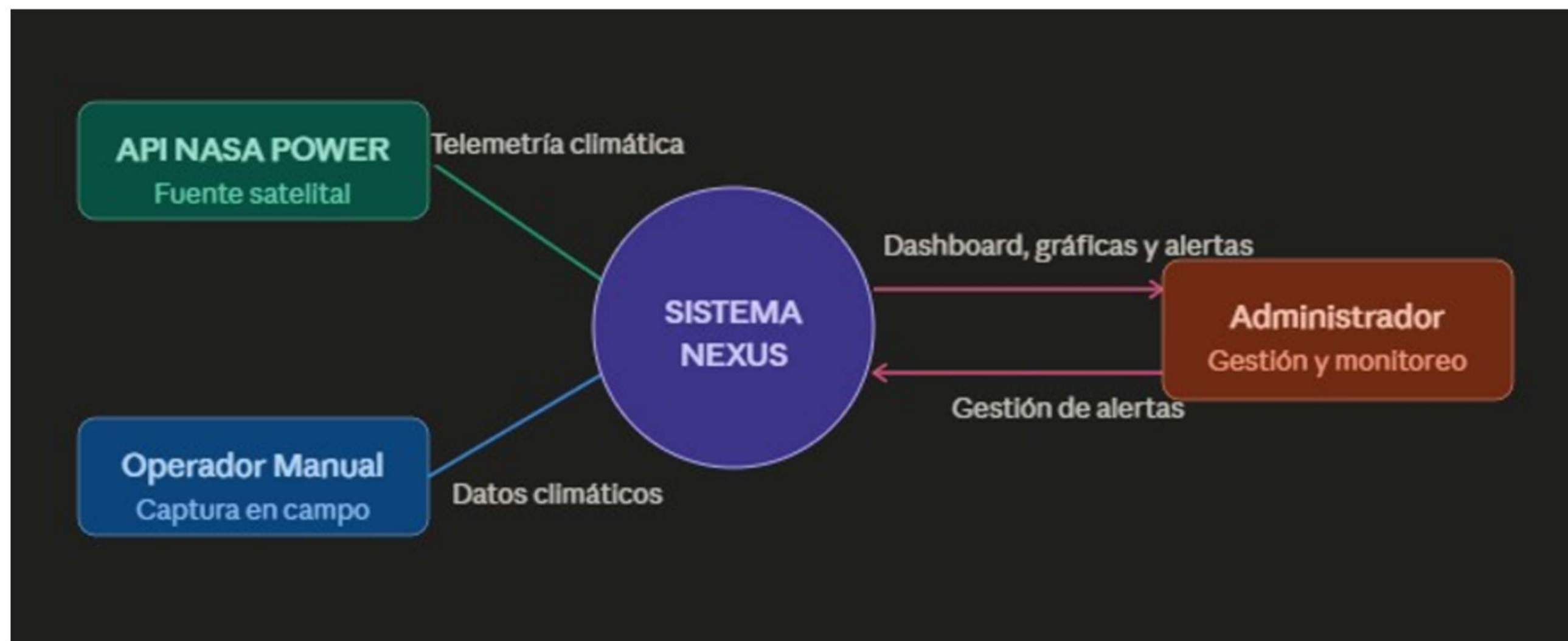
**Sin arquitectura, el código
es solo caos estructurado**



DFD de Contexto

Un Diagrama de Contexto es un modelo conceptual que delimita el alcance de un software, representando al sistema como un único proceso central y detallando los flujos de datos con sus entidades externas.





DFD Nivel 1

El Diagrama de Nivel 1 desglosa el proceso central en sub-procesos específicos, detallando el flujo de información interno y la interacción con los almacenes de datos.



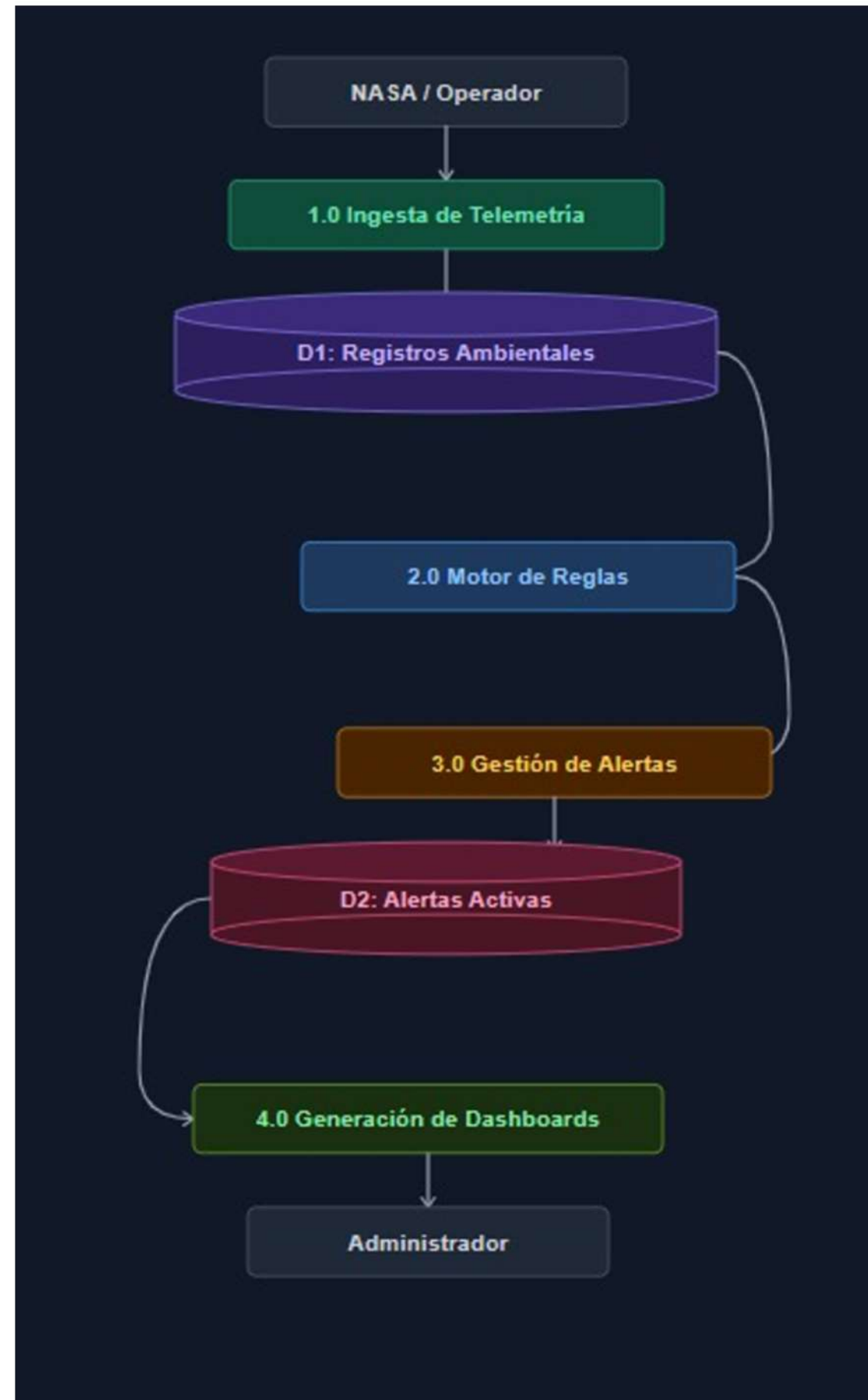
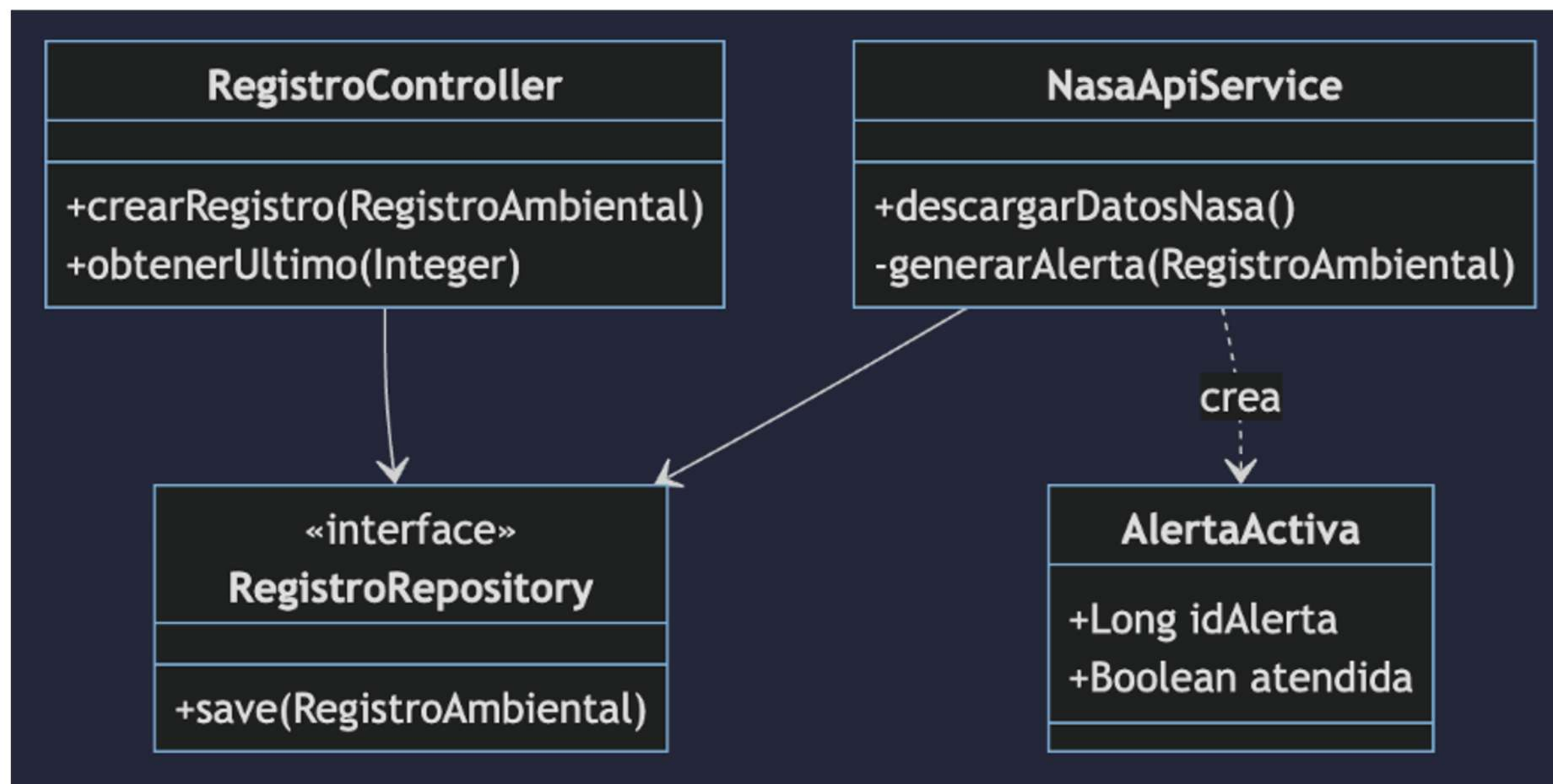


Diagrama UML

El Diagrama de Clases representa la arquitectura orientada a objetos del sistema. Define los componentes lógicos, sus atributos (datos) y sus métodos (funciones), estableciendo cómo se estructurará el código en el backend.





**El software es efímero, los
datos son permanentes**



Modelo ER

El Modelo Entidad-Relación (MER) define la estructura lógica de la base de datos relacional. Traduce los objetos conceptuales del sistema en tablas físicas, asegurando la integridad referencial en el motor MySQL.



Diccionario de Datos

El Diccionario de Datos es el documento de control que describe minuciosamente la estructura interna de cada tabla. Define formalmente los tipos de datos, restricciones y propósitos de cada atributo para evitar la corrupción de la información en MySQL.

Tabla	Atributo	Tipo	Descripción
Zonas	id_zona	INT	Identificador único de cada sector de monitoreo.
Registros	id_registro	BIGINT	Llave primaria de la bitácora climática.
	id_zona	INT	Llave foránea que referencia a la zona.
	fecha_hora	DATETIME	Timestamp de la lectura (hora sincronizada CDMX).
	temp_exterior	DECIMAL	Temperatura medida a 2 metros de altura (°C).
	rad_solar	DECIMAL	Irradiancia solar medida en \$W/m^2\$.
Alertas	id_alerta	BIGINT	Identificador del ticket de anomalía.
	id_registro	BIGINT	Registro que disparó la alerta.
	atendida	BOOLEAN	Estado lógico (0: Activa, 1: Resuelta).

- Persistencia Relacional: Cuatro tablas enlazadas en MySQL que garantizan el almacenamiento de datos climáticos e integridad referencial (ON DELETE CASCADE).
- Automatización en el Motor (Trigger): El código en pantalla se ejecuta de forma síncrona AFTER INSERT. Evalúa la telemetría en microsegundos sin consumir recursos del servidor Java.
- Consumo Dinámico: Cruza los datos en tiempo real de los nuevos registros con los umbrales dinámicos guardados en la tabla de Reglas_Negocio.



```
DELIMITER $$

• CREATE TRIGGER trg_evaluar_reglas_invernadero
  AFTER INSERT ON Registros_Ambientales
  FOR EACH ROW
  BEGIN
    DECLARE lim_temp_int_max DECIMAL(6,2);
    DECLARE lim_hum_suelo_min DECIMAL(6,2);

    SELECT valor_limite INTO lim_temp_int_max FROM Reglas_Negocio WHERE nombre_variable = 'temp_int_max';
    SELECT valor_limite INTO lim_hum_suelo_min FROM Reglas_Negocio WHERE nombre_variable = 'hum_suelo_min';

    IF NEW.temperatura_interior > lim_temp_int_max THEN
      INSERT INTO Alertas_Activas (id_registro, tipo_alerta, mensaje)
      VALUES (NEW.id_registro, 'Temperatura Excesiva', 'Alerta: Umbral superado.');
```


ooo

Pasamos de la teoría
a algo ya aplicado





NASA POWER

Servicio externo que actúa como la fuente de telemetría, proveyendo los vectores climáticos reales mediante payloads en formato JSON.



Spring Boot

Framework en Java 17 encargado de consumir la API externa de forma asíncrona, evaluar los umbrales climáticos y coordinar las acciones del sistema.



MySQL

Motor de base de datos relacional donde se almacena históricamente la información procesada y se disparan las alertas de seguridad.



HTML

Define el esqueleto y la organización de la plataforma, mapeando los contenedores donde se renderizan el Dashboard, las gráficas y los formularios de contingencia.



CSS

Gestiona la identidad visual, el diseño interactivo y el maquetado flexible mediante Grid/Flexbox para que la interfaz se adapte a computadoras, tablets o smartphones.



JavaScript

Controla el comportamiento lógico del navegador. Consume asíncronamente los datos de Spring Boot vía API Fetch y actualiza las gráficas de Chart.js en tiempo real sin recargar la página.

ooo

Probemos el sistema!

ooo

¿Preguntas? ¿Dudas?

